Day 5 - Recurrent Neural Networks

Networks with a memory: RNN, LSTM, GRU

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00 - 10:45	Sequences and memory: the RNN cell • unrolling in time • shapes in PyTorch • vanishing and exploding gradients
11:00 - 12:00	Gates against forgetting: LSTM and GRU • bidirectional and stacked RNNs
13:00 - 14:15	Lab 5: an RNN for heartbeats (task sheet at the end of the PDF)
14:30 - 16:00	Project time

💻 Today we code during the lecture again: open `day5a_rnn` and `day5b_gates` from the course repository in Colab (CPU is enough).

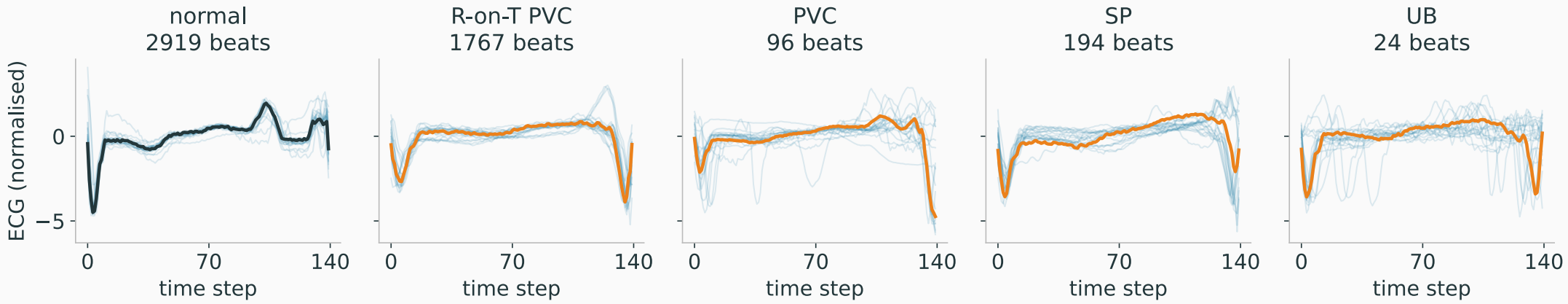
Last time

- **convolution**: a small, shared filter slides over the image - few weights, knows neighbourhoods
- **a CNN**: (conv → ReLU → pool) several times, then flatten → linear; print the shape after every layer
- **the challenge** started: DermaMNIST, balanced accuracy, teams of two

Sequences and memory

The order matters

Sequences are everywhere



- **ECG**: one heartbeat = 140 numbers in a row - the 5 kinds of beats in today's lab
- **EEG**, speech, a sensor in a smartwatch, stock prices - one number (or a few) per time step
- **text**: one word after the other - "the dog bites the man" ≠ "the man bites the dog"

Why not an MLP or a CNN?

- **MLP**: one weight per input position - fixed length, and "the peak at step 10" and "the peak at step 12" are two different things
- **CNN**: a filter slides along the signal - good for local patterns, but each output sees only a **window** (the receptive field)
- what we want: read the signal **step by step** and keep a **memory** of what came before - of any length

❓ Think about it

You read this sentence word by word. What do you keep in your head while you read - and what do you forget?

Your turn: be an RNN

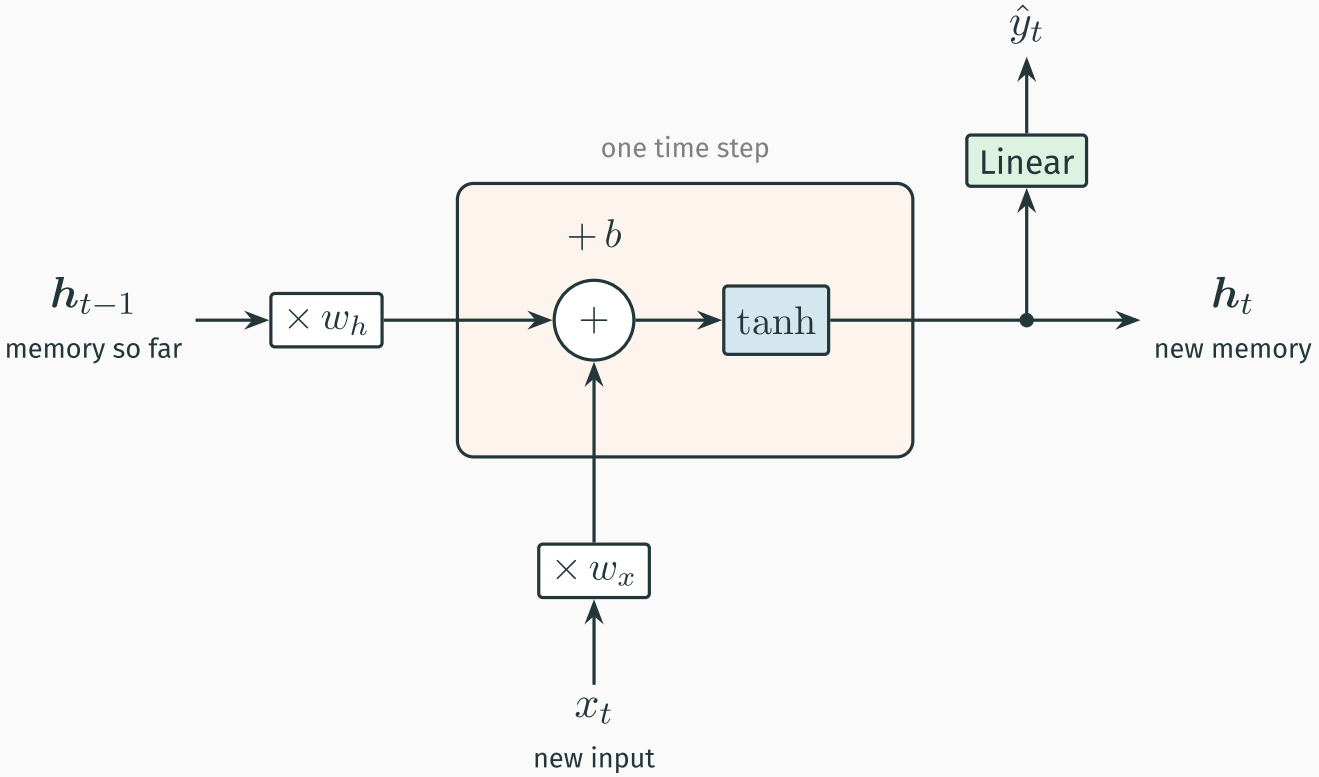
The rule: **new memory = 0.5 · old memory + input**
The memory starts at 0.

step t	**1**	**2**	**3**	**4**
input x	4	0	2	0
memory h	?	?	?	?

👥 In pairs, on paper (6 min)

1. Fill in the memory row.
2. Swap the order: inputs **0, 2, 0, 4**. Is the last memory the same?
3. What does the factor 0.5 do with old inputs?

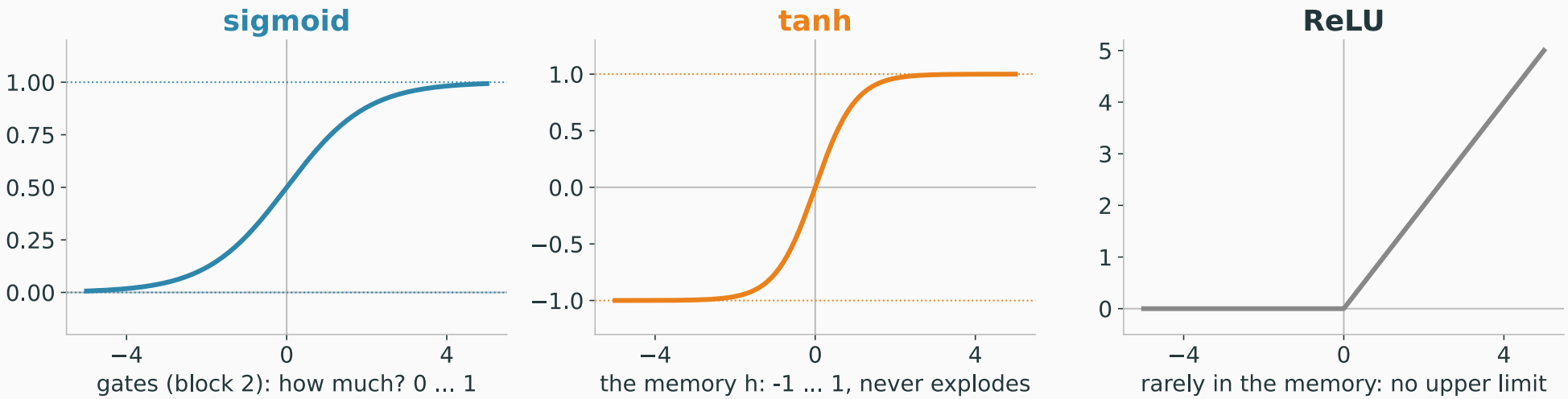
The RNN cell: one small layer, used again and again



new memory = tanh(w_h · old memory + w_x · input + b)

- the same as on paper - with **learned** weights and a **tanh**, so the memory stays between -1 and 1
- the memory h is a vector, e.g. 32 numbers: the **hidden state**
- an output (e.g. a class) comes from h with a normal **Linear** layer

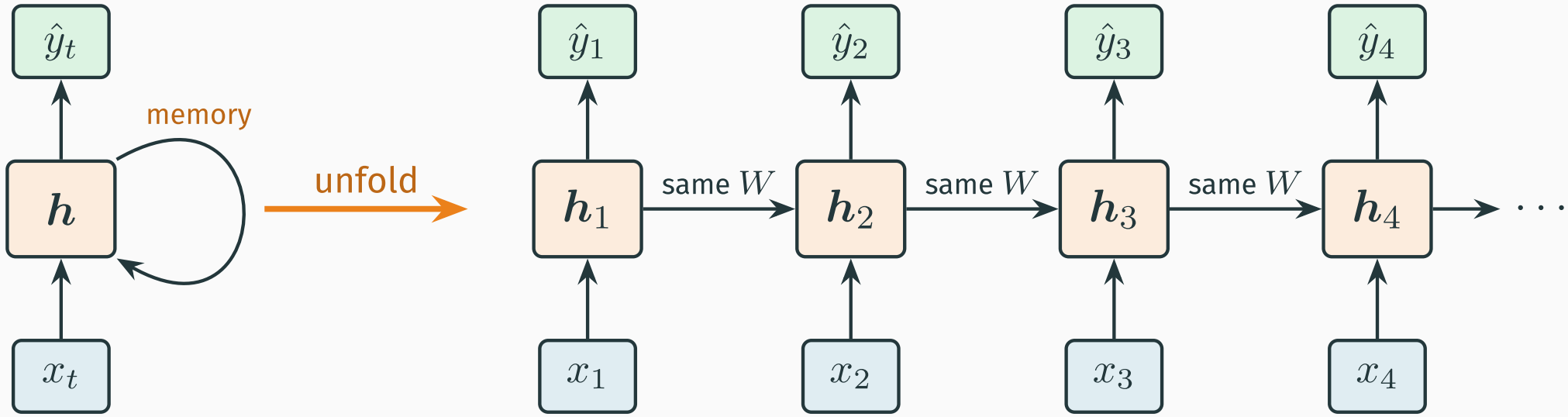
Activation functions in RNNs



where	function	why
the memory h (plain RNN; the candidate in LSTM / GRU)	**tanh**	stays between -1 and 1 - even after 140 steps; can store positive **and** negative values
the gates (LSTM, GRU - block 2)	**sigmoid**	a slider: 0 = closed, 1 = open
the output layer	none - `CrossEntropyLoss` does the softmax	as on days 2-4

`nn.RNN(..., nonlinearity="tanh")` is the default (`"relu"` is possible) • in `nn.LSTM` / `nn.GRU` the functions are fixed

Unrolling in time

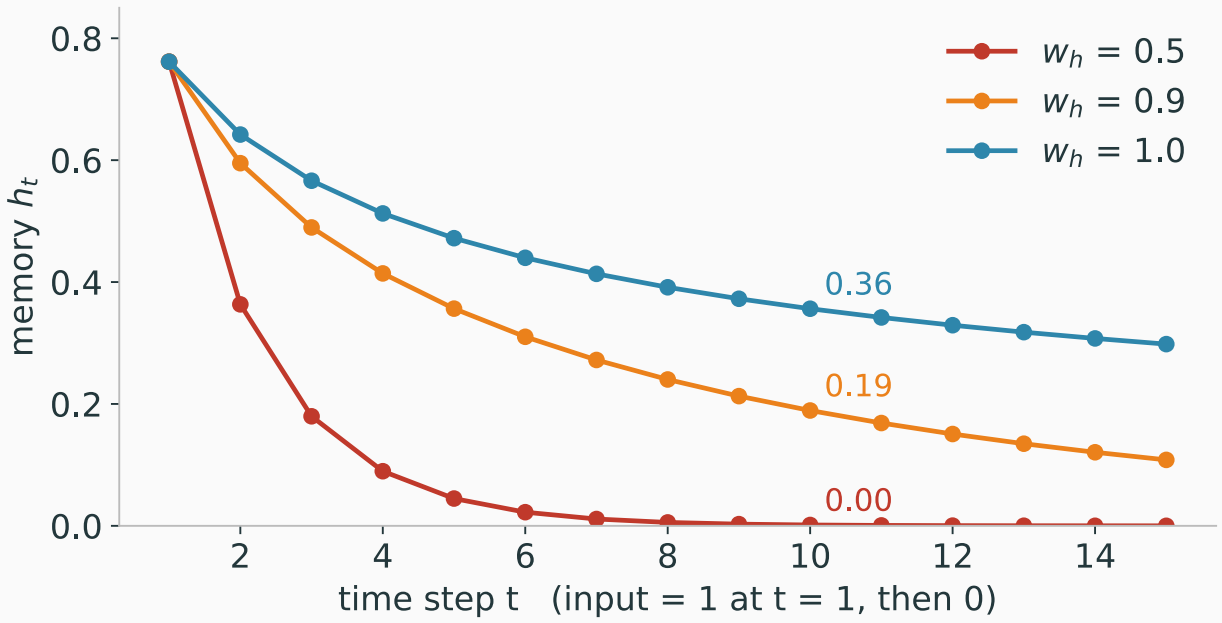


folded

unrolled in time: one copy per time step, all with the same weights

- unrolled, an RNN is a **deep network**: one layer per time step - 140 steps = 140 layers
- but **all copies share the same weights** - like a convolution filter is shared over the image
- the number of weights does **not** depend on the length: one RNN for beats of 140 or 1 000 steps

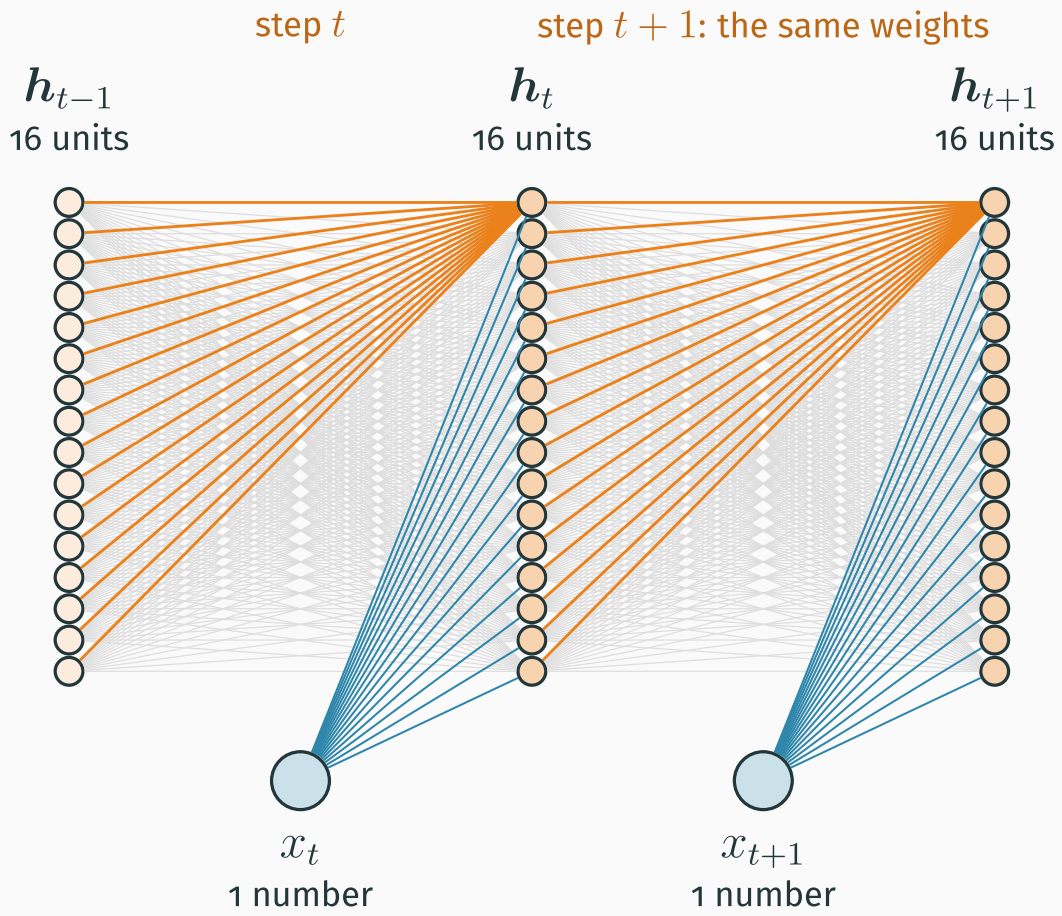
How long does an RNN remember?



Code break 1: input **1** at the first step, then zeros.

- small w_h: the memory halves at every step - after 10 steps it is gone
- larger w_h: it fades more slowly - but it still fades
- very large w_h (2.0): the memory is stuck and can never forget

Inside `nn.RNN(input_size=1, hidden_size=16)`



`hidden_size=16` = **one** layer with **16 memory units** - not 16 layers

weights	
w_h (orange + grey): each new unit looks at all 16 old ones	**256**
w_x (blue): the input goes to every unit	**16**
b: one per unit, PyTorch stores two	**32**
total - the same at **every** step	**304**

❓ Think about it

How many weights with `hidden_size=32`?

In PyTorch: shapes first

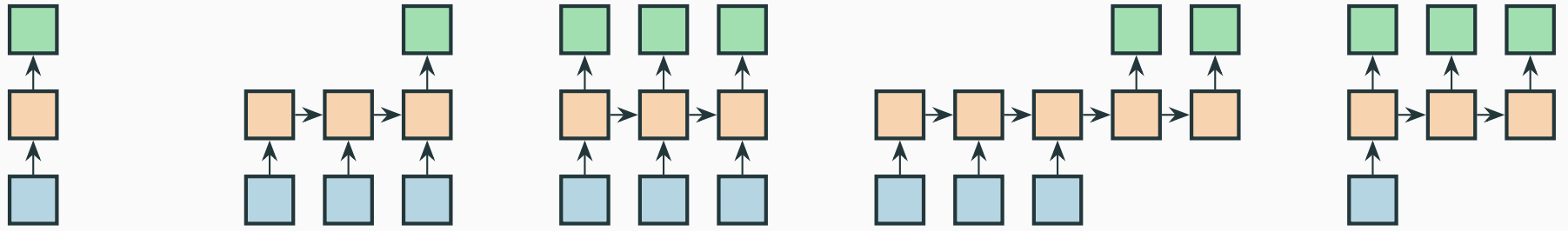
```
beats = X[:8]                       # 8 heartbeats: (batch, time steps, features) = (8, 140, 1)
rnn = nn.RNN(input_size=1, hidden_size=16, batch_first=True)
out, h_last = rnn(beats)            # out: (8, 140, 16) = the memory at EVERY time step
                                    # h_last: (1, 8, 16) = only the last memory
scores = nn.Linear(16, 5)(out[:, -1])     # many-to-one: last memory -> 5 class scores, (8, 5)
```

	shape	meaning
input	(8, 140, 1)	8 beats · 140 time steps · 1 number per step (one ECG lead)
`out`	(8, 140, 16)	the memory (16 numbers) after every time step
`out[:, -1]`	(8, 16)	all beats, **last** time step
parameters	304	w_x 16·1 + w_h 16·16 + two biases 2·16 - for **any** length

❓ Think about it

An EEG with 2 channels, 4-second windows, 256 samples per second, batch of 32: what is the input shape?

One RNN, many kinds of tasks



one-to-one

MLP, CNN
(no memory)

many-to-one

one class per
heartbeat (lab)

many-to-many

one label
per time step

seq2seq

translation

one-to-many

generate text,
music

input memory (hidden state) output arrows to the right: memory is passed on

❓ Think about it

Which picture fits? (a) one sleep stage for every 30 s of a night-long EEG, (b) seizure yes/no for a 4-s EEG window, (c) German → English, (d) predict the next value of a sensor at every step.

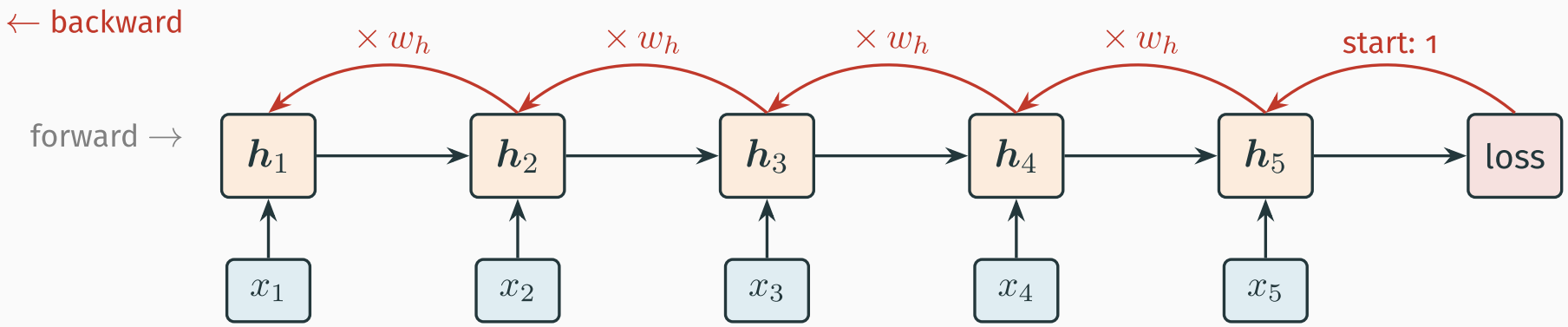
Try it: an RNN in 3D

vizlearn.in/natural_language_processing/rnn.html (A. Jangra, VizLearn)

👥 Your turn (6 min)

In pairs, in your browser. Drag to rotate, scroll to zoom. Match what you see to the pictures of the last slide.

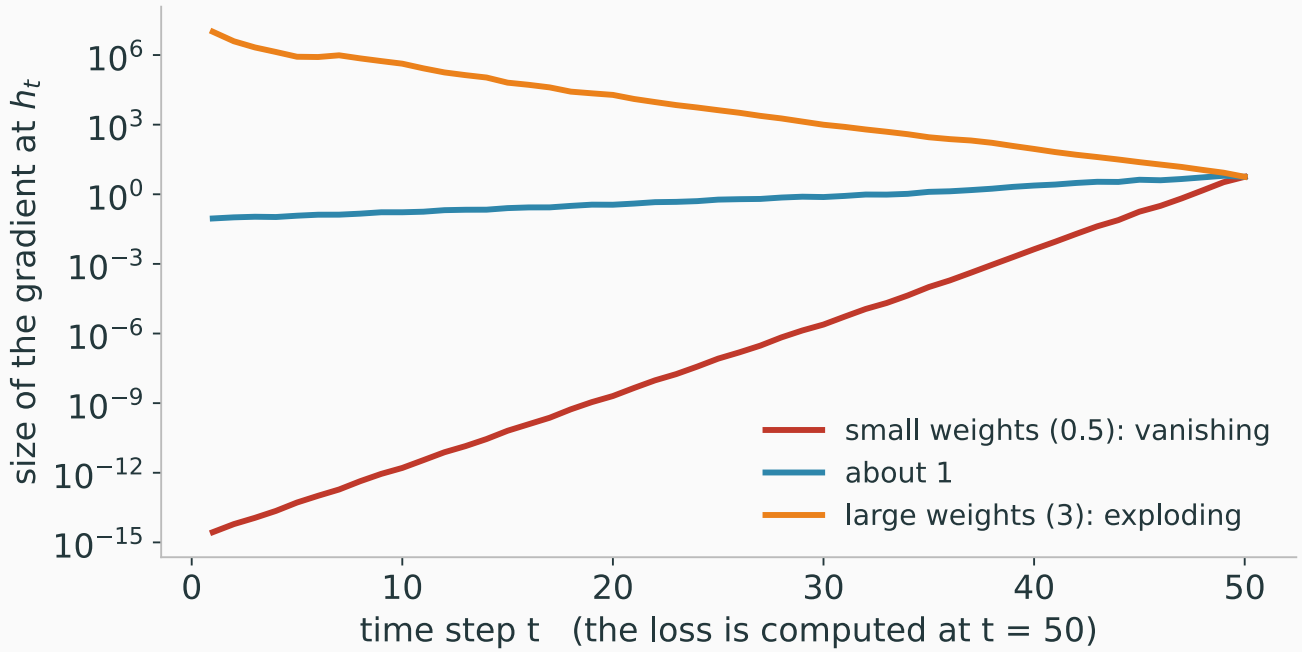
Training: backpropagation through time



gradient with $w_h = 0.5$:	**0.06**	**0.13**	**0.25**	**0.5**	**1**	shrinks: **vanishing**
gradient with $w_h = 1.5$:	**5.1**	**3.4**	**2.3**	**1.5**	**1**	grows: **exploding**

- unrolled, the RNN is a normal network → `loss.backward()` works as on day 2
- the gradient walks **backwards** through the time steps - and is **multiplied by w_h at every step**
- 50 steps back: $0.5^{50} \approx 10^{-15}$ (= 0 for the computer) • $1.5^{50} \approx 6 \cdot 10^{8}$

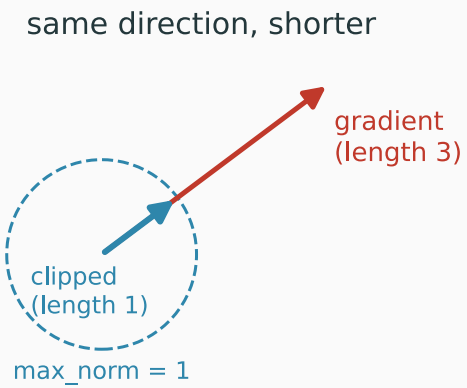
Vanishing and exploding gradients

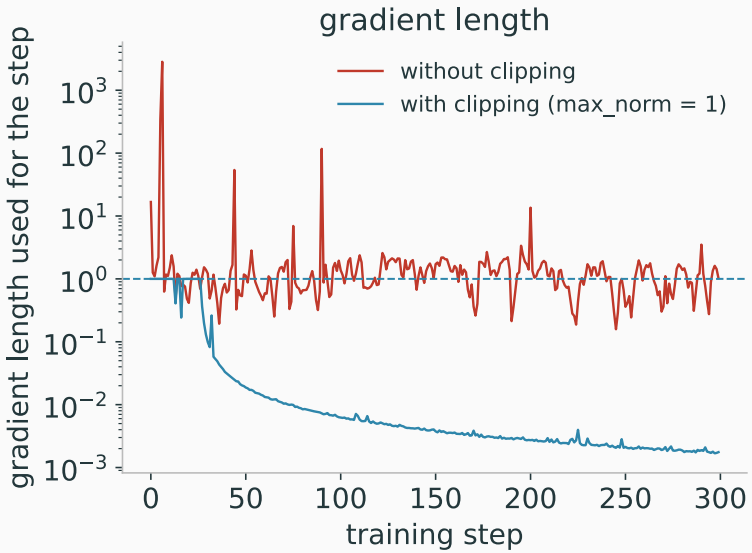


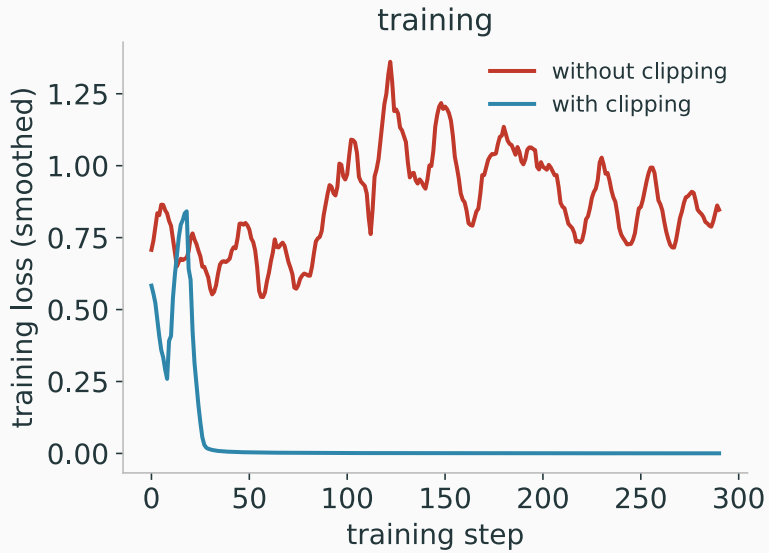
measured in an RNN with 32 memory units, 50 time steps:

- **small** weights: the gradient at step 1 is 10^{-15} - **vanishing** → the RNN cannot learn long memories
- **large** weights: 10^{7} - **exploding** → one step throws the weights far away
- no size is right for every step - plain RNNs remember about **10-20 steps**

Against exploding gradients: gradient clipping








```
loss.backward()
nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
opt.step()
```

- gradient longer than `max_norm` → shrink it, **same direction**
- fixes **exploding** gradients - **vanishing** ones need LSTM / GRU

Cheat sheet: RNNs in one slide

idea	read the sequence step by step, keep a memory h (hidden state)
one step	new memory = tanh(w_h · old memory + w_x · input + b) - the **same** weights at every step
PyTorch	`nn.RNN(input_size, hidden_size, batch_first=True)`, input `(batch, time, features)`
outputs	`out` = memory at every step (many-to-many) • `out[:, -1]` = last memory (many-to-one)
training	backpropagation through time: `loss.backward()` on the unrolled graph
problem	the gradient is multiplied at every step → **vanishing** (no long memory) or **exploding**
fix 1	gradient clipping: `clip_grad_norm_(params, 1.0)` between `backward()` and `step()`
fix 2	gates: LSTM and GRU → after the break

Gates against forgetting: LSTM and GRU

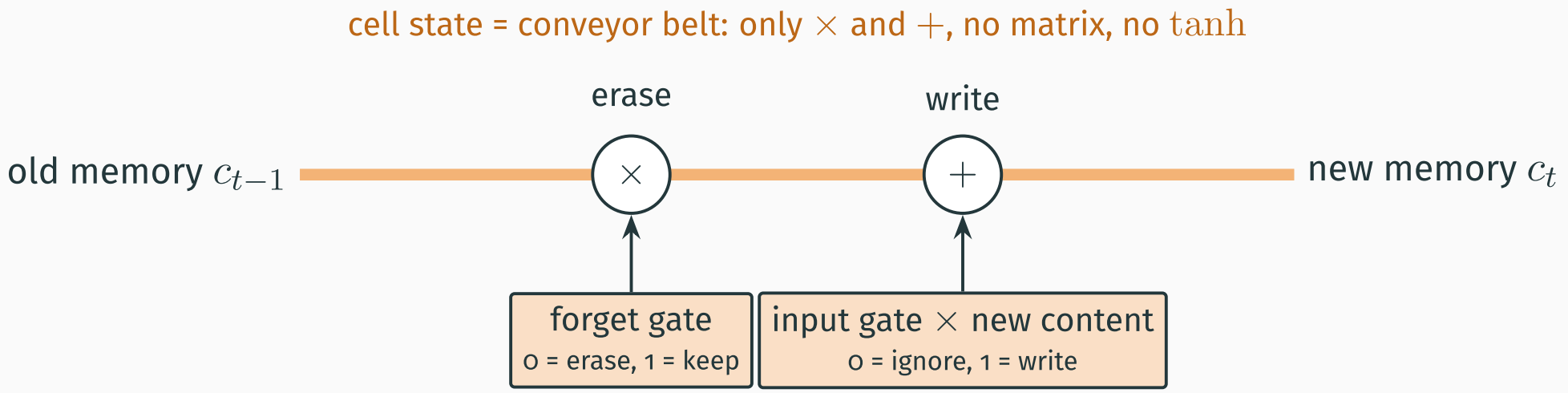
A memory that can decide to keep

A test for memory: remember the first bit

input (T steps): **+1**, -1, -1, +1, -1, +1, +1, -1, … , +1
question at the end: **what was the first bit?** → +1

- every step brings a new random bit that has nothing to do with the answer
- to solve it, the network must **store one bit** and **protect it** for T steps
- guessing = 50 %

The LSTM idea: a conveyor belt for memory



- plain RNN: the memory is **rewritten** at every step (weights × memory, then tanh) → it fades
- LSTM: an extra memory, the **cell state** c - it is only **multiplied by a gate** and **added to**
- forget gate = 1 and input gate = 0 → the memory rides along **unchanged**, for as long as needed

Gates are sliders between 0 and 1

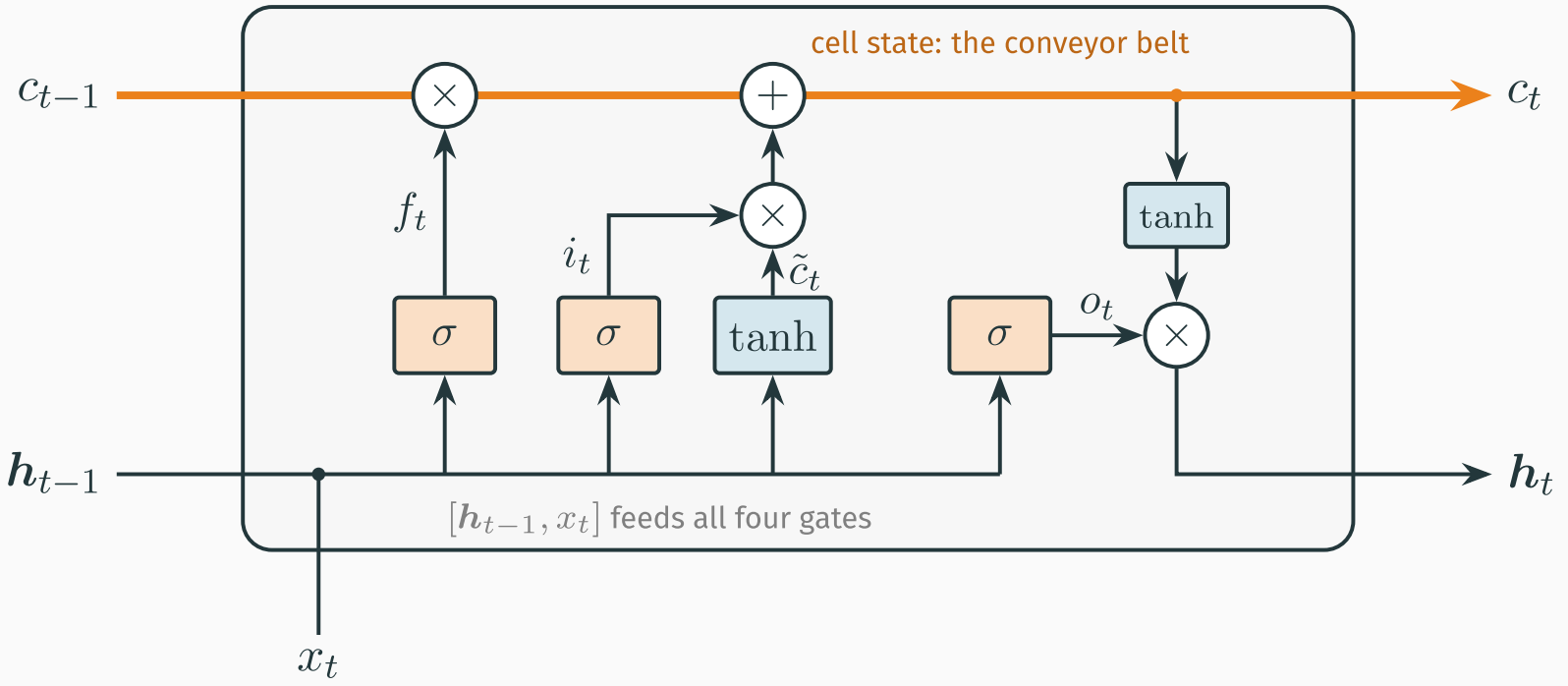
gate	≈ 0	≈ 1
forget	erase the memory	keep it
input	ignore this input	write it
output	keep the memory private	show it to the next layer

- each gate is a small layer with a **sigmoid** (day 2): its output is between 0 and 1
- it looks at the current input and the last memory - the network **learns** when to open and close

❓ Think about it

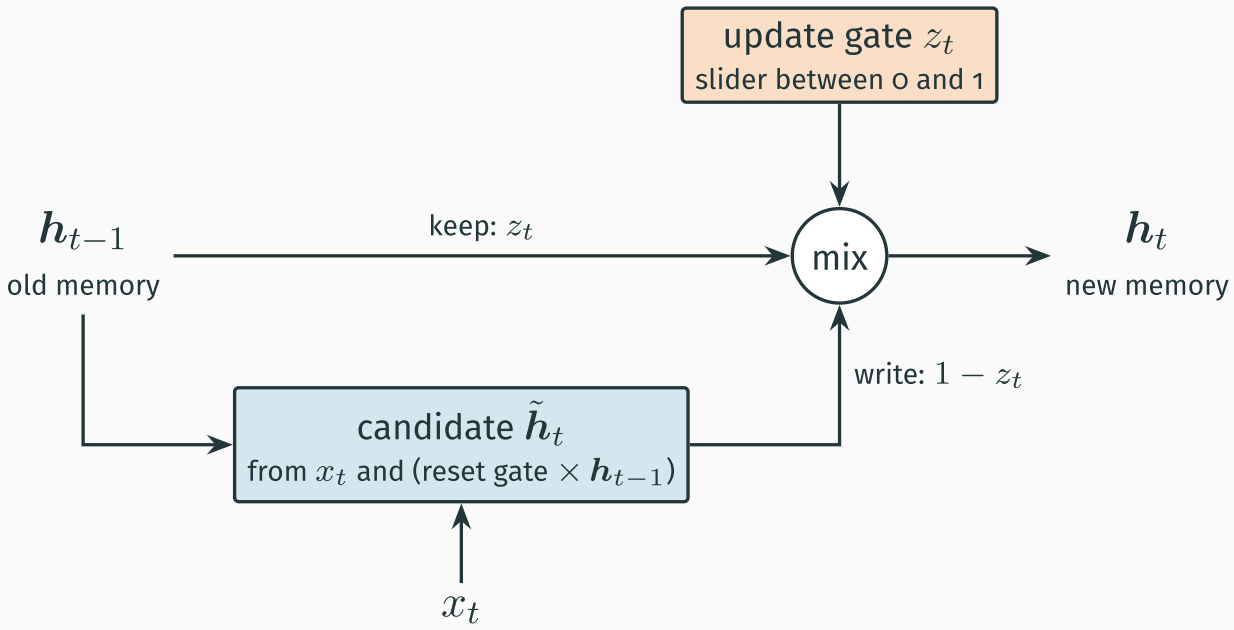
What does an LSTM do if the forget gate is always 1 and the input gate always 0? And if the forget gate is always 0?

The full LSTM cell - you do not need to memorise it



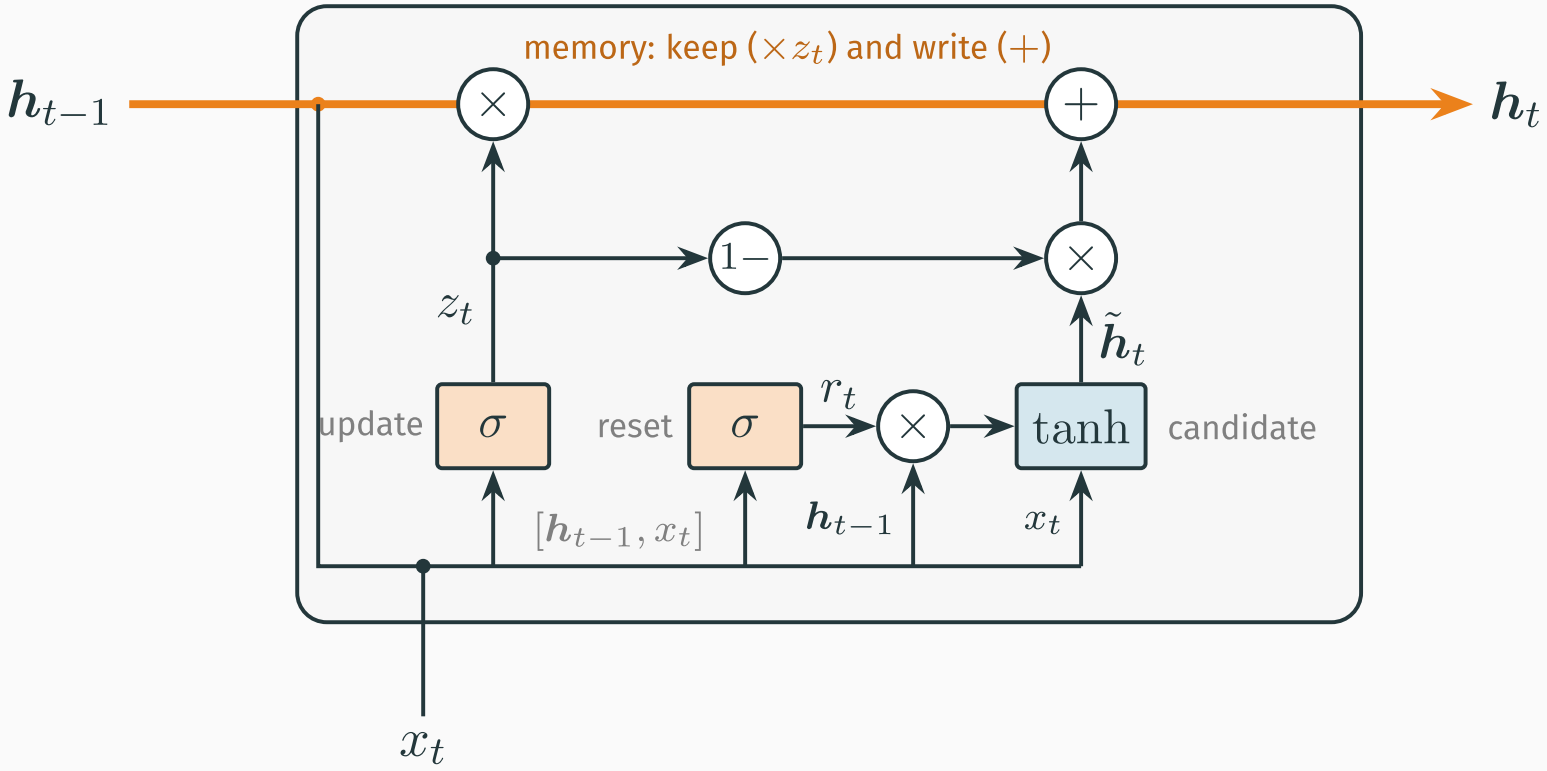
- top line: the **conveyor belt** c - only × (forget) and + (write)
- 3 orange gates (σ, between 0 and 1) and 1 blue candidate (tanh): **4 small layers** → 4 × the weights of a plain RNN
- the output h is a filtered view of the belt (output gate)

The GRU: a lighter gated cell



- **one** memory h, no extra belt
- the **update gate** z is a slider: keep the old memory (z = 1) or write the new candidate (z = 0) - forget and input in **one** gate
- **reset gate**: how much of the old memory the candidate may look at
- 3 small layers → 3 × the weights of a plain RNN

The full GRU cell - you do not need to memorise it



- top line: the memory h - only **×** z_t (keep) and **+** (write), like the LSTM belt
- **update gate** z_t: keep the old memory with z_t, write the candidate with $1 - z_t$
- **reset gate** r_t: how much of the old memory the candidate $\tilde{h}_t$ may use
- 2 orange gates (σ) + 1 blue candidate (tanh): **3 small layers** → 3 × the weights of a plain RNN

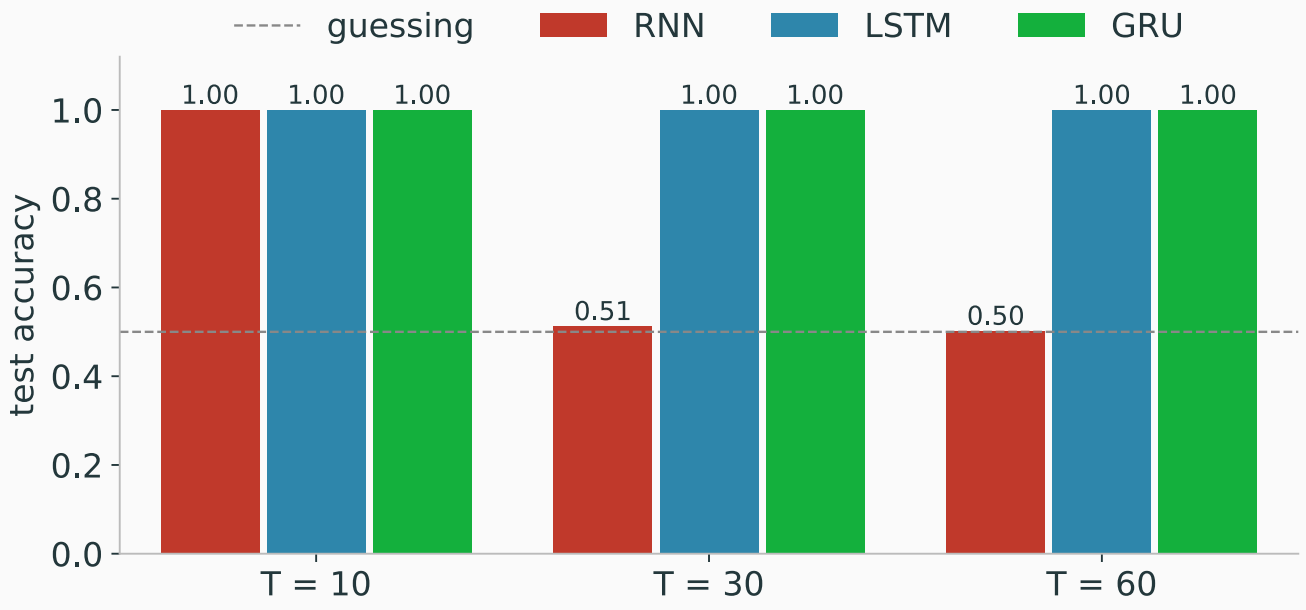
In PyTorch: one word changes

```
rnn  = nn.RNN(input_size=1, hidden_size=16, batch_first=True)     #   304 parameters
gru  = nn.GRU(input_size=1, hidden_size=16, batch_first=True)     #   912 = 3 ×
lstm = nn.LSTM(input_size=1, hidden_size=16, batch_first=True)    # 1 216 = 4 ×

out, _ = lstm(beats)        # out: (8, 140, 16) - the same as for RNN and GRU
                            # _ = (h, c): the LSTM gives back TWO memories
```

- same input, same `out` - the rest of your model stays the same
- the only trap: `nn.LSTM` returns `(h, c)` instead of `h` → write `out, _ = ...`

Remember the first bit: the result

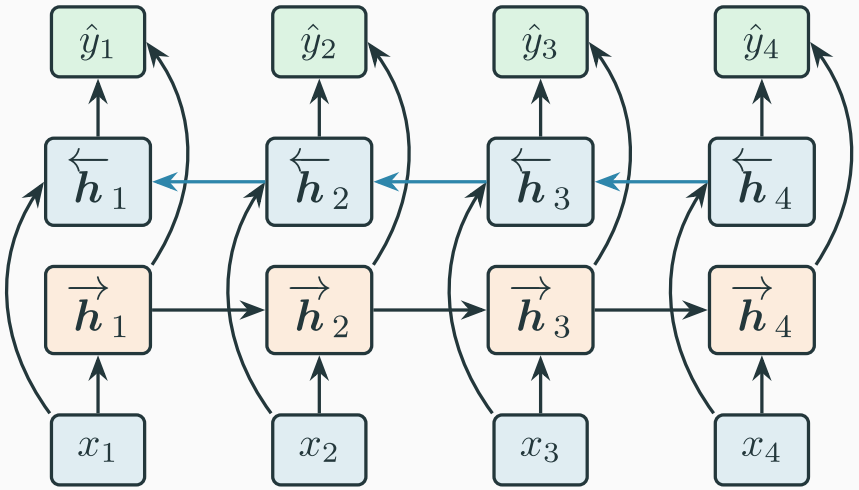


- the plain RNN only solves **T = 10** - from 30 steps on it **guesses**
- LSTM and GRU solve all lengths - each run takes a few seconds
- one trick inside: the "keep" gate starts at **remember** (bias + 3)

🔑 Key takeaway

Gates make a long memory **possible** - and trainable.

Bidirectional: read forwards and backwards



output uses $[\overrightarrow{h}_t; \overleftarrow{h}_t]$

backward: reads $x_T \to x_1$

forward: reads $x_1 \to x_T$

- a second RNN reads the sequence **from the end** - every output sees the past **and** the future
- `nn.GRU(..., bidirectional=True)`: `out` has **2 ×** the numbers, 2 × the weights
- good when the whole sequence is there: a finished heartbeat, a sentence, a recorded night

Stacked RNNs - and a CNN in front

stacked: `nn.LSTM(..., num_layers=2)`

- the second layer reads the memories of the first
- 2-3 layers are typical; `dropout=0.2` between them

CNN + RNN: Conv1d first, then a GRU

- the CNN finds local patterns and makes the sequence **shorter** (pooling)
- the GRU combines them over a long time

Which one when?

	plain RNN	GRU	LSTM	1D CNN (day 4)
memory	short (10 - 20 steps)	long	long	only its window
weights (vs RNN)	1 ×	3 ×	4 ×	-
speed	step by step	step by step	step by step	**all steps at once**
use it as	baseline, teaching	small data, devices	default	strong, fast baseline

- GRU vs LSTM: in big comparisons **no clear winner** - try both, it is one word
- a CNN is the fast baseline for short signals - always compare with it (lab, part 4)

Outlook: attention

- an RNN must squeeze everything into **one memory vector** - long sequences are still hard
- **attention**: at every step, look back at **all** earlier steps directly and choose what matters - no conveyor belt needed
- the **Transformer** (2017) uses only attention - no recurrence, all steps in parallel: ChatGPT, Whisper, EEG foundation models
- **day 6**: Transformers - and transfer learning: use a big pre-trained model for your own data

Lab 5: an RNN for heartbeats

13:00-14:15 · task sheet at the end of the PDF

Lab 5: write it yourself - RNN, GRU, LSTM on ECG beats

💻 `day5_rnn.ipynb` in Colab · CPU is enough · download: 10 MB

Part 1 - look at the data (~10 min)

- 1.1 five kinds of heartbeats - very imbalanced
- 1.2 shapes for an RNN

Part 2 - your RNN classifier (~20 min)

- 2.1 write `ECGRNN(cell_type)`: one class for RNN, GRU and LSTM

Part 3 - train it (~15 min)

- 3.1 accuracy vs balanced accuracy
- 3.2 class weights

Part 4 - one table, four models (~20 min)

- 4.1 RNN vs GRU vs LSTM vs 1D CNN
- 4.2 where does the best model fail?

👥 How it works

In pairs. Plan → write the code → 💡 hint → ✅/❌ check. Not finished = homework.

Challenge: project work

14:30-16:00

Project time

- **free work** in your teams on the challenge
- questions? I come round to every team
- if you like: we look at a **paper** together - an idea for your model

Dates

- **10 Nov**: send your model (`model.pt`)
- **11 Nov**: print the poster (at the latest) · poster presentations · leaderboard
- **15 Nov**: hand in on **CampusWeb**: link to the code, model, report (PDF), poster (PDF/PNG), app

Wrap-up

Summary

- **RNN**: the same small layer, step by step, with a memory h - any length, few weights
- **shapes**: `(batch, time, features)` with `batch_first=True`; `out[:, -1]` for many-to-one
- **BPTT**: the gradient is multiplied at every step → vanishing / exploding; clipping fixes exploding
- **LSTM / GRU**: gates (sliders 0-1) protect the memory → long memories become learnable
- **bidirectional**: a second RNN reads backwards - every output sees past and future
- **imbalanced classes**: balanced accuracy + confusion matrix + class weights

📄 Everything on one page: Stanford CS230 cheat sheet: recurrent neural networks (parts Overview and Handling long term dependencies)

Lab 5 – Write it yourself: an RNN for heartbeats

Notebook: `day5_rnn.ipynb` (course repository) · open it in Colab · CPU is enough · work in **pairs** · what you do not finish in class is **homework**.

How every writing task works

1. read the **plan** (a small table on this sheet and in the notebook),
2. write your code below the numbered steps – where it says `pass  # YOUR CODE HERE`,
3. stuck? open **Hint** in the notebook,
4. run the **check cell**: ✔ = correct, ✘ = it tells you what is wrong,
5. tick the box ☐ of the task on this sheet.

First run the two **setup cells**. Something looks strange? *Runtime* → *Restart session*, then the setup cells again.

Data: ECG5000 – 5 000 heartbeats, each **140 time steps**, already normalised. 5 classes: 0 normal · 1 R-on-T PVC · 2 PVC · 3 SP · 4 UB (unclassified). Split 70 / 15 / 15 % into training, validation and test.

Part 1 – Look at the data — about 10 min

☐ 1.1 Five kinds of heartbeats — 5 min

Run the plot: 20 training beats per class.
Question: which classes look similar? How many training beats do the two rarest classes have?

☐ 1.2 Shapes for an RNN — 5 min

An RNN layer gets the data as a block with three sizes, always in this order: `(batch, time steps, features)`. Run the cell: a batch of 8 beats has the shape `(8, 140, 1)`.
Question: what does each of the three numbers mean? What would the last number be for an EEG with 2 channels?

Part 2 – Your RNN classifier — about 20 min

☐ 2.1 Write `ECGRNN(cell_type)` – one class for RNN, GRU and LSTM — 20 min

Many-to-one: the recurrent layer reads the beat, a linear layer turns the **last** memory into 5 scores. `cell_type` is the text `"RNN"`, `"GRU"` or `"LSTM"`, e.g. `ECGRNN("GRU")`.

1 `if cell_type == "RNN":`
`self.rnn = nn.RNN(input_size=..., hidden_size=..., batch_first=..., bidirectional=...)`
2 the same with `elif cell_type == "GRU":` → `nn.GRU` and `elif cell_type == "LSTM":` → `nn.LSTM`
3 `self.out = nn.Linear(size, 5)` (`size` is given: 32, or 64 if bidirectional)
4–5 `out, _ = self.rnn(x)`, then `scores = self.out(out[:, -1])` (last time step)

Check: output `(8, 5)`; parameters RNN 1 285 · GRU 3 525 · LSTM 4 645 · GRU bidirectional 7 045.
Question: why does the number of parameters **not** depend on the 140 time steps?

Part 3 – Train it — about 15 min

☐ 3.1 A GRU – and two scores — 8 min

Train `ECGRNN("GRU")` with the given `train_model` (20 epochs, about 30–60 s) and look at the confusion matrix.
Question: why is the accuracy so much higher than the balanced accuracy?

☐ 3.2 Class weights — 7 min

Train again with `weighted=True`: mistakes on rare classes cost more in the loss.

	accuracy	balanced accuracy
3.1 without weights		
3.2 with weights		

Question: is this a good trade – and who decides?

Part 4 – RNN vs GRU vs LSTM vs CNN — about 20 min

☐ 4.1 One table, four models — 15 min

Write the loop: `for name, model in models.items():` train every model with class weights and store the scores in `results[name]`. The 1D CNN of day 4 is given.

model	accuracy	balanced accuracy	parameters	seconds
RNN				
GRU				
LSTM				
CNN				

Question: which model is best – which is fastest? Why is the plain RNN worse than GRU and LSTM?

☐ 4.2 Where does the best model fail? — 5 min

Confusion matrix of your best model. **Question:** which two classes are confused most often – and why?

☐ ★ Bonus Bidirectional, bigger memory, test set — homework

(1) `ECGRNN("GRU", bidirectional=True)` – better? Why 64 inputs for the linear layer? (2) `ECGRNN("LSTM", hidden=64)`. (3) Choose your final model on the validation scores, **then** evaluate it **once** on the test beats.